

Architektur- und Datenstruktur-Konzept

Offline-First Vokabel-App mit Sembast & Flutter

Technisches Konzept

10. September 2026

Dieses Dokument fasst die Design-Entscheidungen für eine robuste, Sembast-basierte lokale Datenbank zusammen, die für eine zukünftige, fehlerfreie Synchronisation (z. B. via Google Drive) und den CSV-Austausch vorbereitet ist. Der Fokus liegt auf der Vermeidung von Datenverlusten durch Offline-Konflikte bei gleichzeitig pragmatischer Komplexität.

1 Das Datenbankschema (Sync-Ready)

Um die Daten über mehrere Geräte hinweg eindeutig identifizieren und mergen zu können, werden UUIDs anstelle von lokalen Auto-Increments eingesetzt. Das Schema gliedert sich in drei Entitäten.

1.1 Vokabelbuch (Book)

Dient als oberster Container und steuert die Synchronisations- und Bereinigungs-Metadaten.

Feld	Typ	Beschreibung
id	UUID	Globale, eindeutige Kennung.
name	String	Anzeigename des Buches.
modified_at	Timestamp	Der Sync-Türsteher: Ändert sich bei <i>jeder</i> Aktion im Buch (Inhalt, Löschen, Sortieren). Dient dem schnellen Vorabgleich und bestimmt den Gewinner bei Sortier-Konflikten.
cleaned_at	Timestamp	Die Epoche: Zeitpunkt der letzten Garbage Collection (physisches Löschen der Tombstones). Verhindert das Zombie-Problem alter Offline-Stände.

1.2 Kapitel (Chapter)

Strukturiert Vokabeln und unterliegt denselben Synchronisationsregeln wie die Vokabeln selbst.

Feld	Typ	Beschreibung
id	UUID	Globale ID.
book_id	UUID	Referenz zum übergeordneten Vokabelbuch.
name	String	Name des Kapitels.
sort_order	Integer	Position des Kapitels im Buch (1, 2, 3 ...).
created_at	Timestamp	Erstellungsdatum (zur Erkennung neuer Offline-Datensätze).
updated_at	Timestamp	Dokumentiert <i>ausschließlich</i> inhaltliche Änderungen (Name, Lösch-Status).
is_deleted	Boolean	Tombstone-Flag (Soft Delete).

1.3 Vokabel (Vocabulary)

Hält den eigentlichen Lerninhalt und trennt strikt zwischen inhaltlicher Mutation und reiner Positionsinformation.

Feld	Typ	Beschreibung
id	UUID	Globale ID.
chapter_id	UUID	Referenz zum Kapitel. Eine Änderung hier gilt als strukturelle/inhaltliche Mutation.
lang1 / lang2	String	Vokabeltexte und Übersetzungen.
comment	String	Optionalen Kommentar.
sort_order	Integer	Position innerhalb des Kapitels (1, 2, 3 ...).
created_at	Timestamp	Erstellungsdatum.
updated_at	Timestamp	Dokumentiert <i>ausschließlich</i> inhaltliche Änderungen (Texte, chapter_id, is_deleted).
is_deleted	Boolean	Tombstone-Flag (Soft Delete).

2 Die Logik der Mutationen (Zeitstempel-Handling)

Entscheidend für das System ist die formale Trennung zwischen *Inhalt* und *Sortierung*:

- **Inhaltliche Änderung:** Korrektur eines Textes oder Umbenennung eines Kapitels.
→ updated_at des Elements wird aktualisiert; modified_at des Buches wird auf denselben Zeitpunkt gesetzt.
- **Strukturelle Zuordnung:** Verschieben einer Vokabel von Kapitel A nach Kapitel B.
→ Gilt als inhaltliche Mutation (chapter_id ändert sich). updated_at der Vokabel und modified_at des Buches werden aktualisiert.
- **Reine Sortier-Änderung:** Verschieben einer Vokabel innerhalb desselben Kapitels.
→ Die sort_order-Integers der betroffenen Einträge werden neu vergeben. Das updated_at der Vokabeln bleibt **unverändert**. Lediglich modified_at des Buches wird aktualisiert.

3 Lösch-Logik (Tombstones & Cascading Deletes)

Um Synchronisationskonflikte zu beherrschen, werden Daten standardmäßig nicht sofort physisch entfernt:

- **Soft Deletes:** Beim Löschen wird `is_deleted = true` gesetzt und `updated_at` aktualisiert. Die App filtert diese Datensätze in UI und CSV-Export heraus.
- **Cascading Soft-Deletes:** Wird ein Kapitel gelöscht, müssen zwingend alle zugeordneten Vokabeln ebenfalls `is_deleted = true` und ein neues `updated_at` erhalten, um verwaiste Einträge (*Orphans*) zu vermeiden.
- **Garbage Collection (Zukunft):** Bei einer aktiven Bereinigung werden Tombstones physisch gelöscht und das Buch erhält ein `cleaned_at`. Reicht ein Client später einen Sync-Stand ein, der älter als `cleaned_at` ist, werden dessen veraltete Tombstones ignoriert (Schutz vor dem Zombie-Problem).

4 Konfliktauflösung & Merge-Strategie

Beim Zusammenführen zweier Offline-Stände greift eine zweistufige Priorisierung:

1. **Inhaltliche Ebene (`updated_at`):** Für `lang1`, `lang2`, `comment`, `is_deleted` und `chapter_id` gewinnt pro Datensatz der neuere `updated_at`-Zeitstempel.
2. **Strukturelle Ebene (`modified_at` & LWW):** Das Vokabelbuch mit dem jüngeren `modified_at` bestimmt die Reihenfolge aller Kapitel und Vokabeln (*Last-Write-Wins*). Offline hinzugefügte Neuzugänge des älteren Buches werden heuristisch (nach einer gemeinsamen Anker-Vokabel) oder am Ende des Kapitels angehängt. Anschließend wird das Kapitel lokal mit sauberen Integer neu nummeriert.

Exkurs zu Fractional Indexing: Die Verwendung von reellen Zahlen (1.5, 2.25) oder lexikografischen Strings (wie bei Jira LexoRank) zur kollisionsfreien Sortierung wurde bewertet, für die gegebene Größenordnung (< 100 Vokabeln pro Kapitel) jedoch zugunsten von Integer und Neunummerierung als Overengineering verworfen.

5 Entwicklungsphasen

1. **Phase 1 (Vorbereitung):** Sembast-Datenbankmodelle exakt nach diesem Schema aufbauen. UU-IDs, Zeitstempel und Tombstones von Tag 1 an mitführen.
2. **Phase 2 (Manuell):** CSV-Import und -Export implementieren mit Dialogen zur Nutzerentscheidung (*Neues Buch, Überschreiben, Zusammenführen*).
3. **Phase 3 (Auto-Sync):** Spätere Anbindung von Google Drive mit dem definierten Merge-Algorithmus ohne Notwendigkeit einer Schema-Migration.